EXPERIMENT 04

Frequency Domain Image Filtering

Fourier transform, filter design, reconstruction and measured analysis

BTCS301P - AS04

Complete laboratory solution with executable Python, an executed notebook, reproducible results and answers to all ten questions.

DOCUMENT CONTENTS

Part I - Aim, theory, algorithm and implementation
Part II - Measured observations and comparison figures
Part III - Complete answers to Questions 1-10
Appendix A - Runnable reference program
Appendix B - Verification and GitHub submission

Student identity fields are intentionally omitted. Add the details required by your institution before submission.

Experiment 4: Frequency Domain Image Filtering using Fourier Transform

Course: BTCS301P | Assignment: AS04 | Implementation: Python

1. Aim

Implement image filtering in the frequency domain using the two-dimensional Discrete Fourier Transform (DFT). Analyze how low-pass and high-pass transfer functions affect noise, smoothness, edges, and fine texture. Reconstruct filtered images using the inverse DFT, compare numerical and visual results, and explain the computational and practical trade-offs.

2. Scope and deliverables

The required workflow is implemented in full. The minimum experiment uses a Gaussian low-pass filter and its complementary high-pass filter. Ideal and Butterworth filters extend the comparison to transition sharpness and ringing. The repository also includes a cutoff study, numerical verification, a CLI for external images, an executed notebook, and detailed answers to all ten questions.

This is an educational image-processing experiment. Results establish behavior on the specified synthetic image and noise realization. They do not establish performance on a clinical, satellite, biometric, or industrial benchmark.

3. Software and data

Python 3.11 or 3.12, NumPy, OpenCV, and Matplotlib implement the experiment. Pytest verifies numerical properties; nbformat/nbclient execute the notebook; ReportLab creates the printable solution. Exact locally tested versions and commands are recorded in the repository.

The generated 256 x 256 grayscale reference contains a smooth background, rectangle, disk, narrow diagonal line, smooth bright region, and sinusoidal texture. It provides both slowly varying regions and abrupt transitions. The reference is rounded to 8-bit intensity values. Seeded zero-mean Gaussian noise with standard deviation 20 intensity units is added, then clipped and rounded to simulate an 8-bit observation. Consequently the final noise is only approximately Gaussian near intensity limits, and its measured MSE need not equal 20 squared exactly.

The notebook saves the noisy image as a lossless PNG and loads it with `cv2.imdecode(..., cv2.IMREAD_GRAYSCALE)` through `load_grayscale`. This explicitly performs the assignment's grayscale-image loading step. The clean image is used only for controlled evaluation, never supplied to a filter.

4. Mathematical foundation

4.1 Spatial frequency

An image f[x,y] records intensity at a row and column. A slowly varying background has predominantly low spatial frequency. Rapid changes, including object boundaries, fine texture, and some kinds of noise, contribute higher frequencies. A two-dimensional frequency has a horizontal and vertical component. Its direction describes intensity variation; stripes are perpendicular to that variation. Noise is not inherently confined to high frequencies.

4.2 Forward and inverse DFT

For M rows and N columns, the convention used here is:

```
F[u,v] = sum(x=0..M-1) sum(y=0..N-1)
         f[x,y] * exp(-j*2*pi*(u*x/M + v*y/N))

f[x,y] = (1/(M*N)) * sum(u=0..M-1) sum(v=0..N-1)
         F[u,v] * exp(+j*2*pi*(u*x/M + v*y/N))
```

Here j squared is -1. Each coefficient is complex: its magnitude describes the strength of a sinusoidal component, and its phase records relative alignment. The FFT is an efficient algorithm for the DFT, not a different transform. NumPy's default forward transform is unscaled, while its inverse divides by M times N. These conventions are documented in the NumPy FFT reference.

The DC coefficient F[0,0] equals the sum of all pixel intensities; F[0,0]/(M*N) is the mean. For a real image, conjugate frequency pairs have equal magnitudes and opposite phases. Preserving the appropriate symmetry during filtering produces a real-valued inverse result up to floating-point error.

4.3 Centering and spectrum display

`fft2` stores DC at index (0,0). `fftshift` rearranges coefficients so DC is at (M//2,N//2), with nearby low frequencies around it. This is a permutation; it does not increase resolution, change amplitudes, or remove noise. Apply `ifftshift` before the inverse transform. The inverse shift matters especially for odd-sized arrays. See the official fftshift API.

Display the centered magnitude using:

```
S[u,v] = ln(1 + abs(F_centered[u,v]))
```

The logarithm compresses the large dynamic range; adding one avoids log(0). This quantity is neither a power spectrum nor a calibrated decibel spectrum. The complex, unlogged coefficients are retained for filtering and reconstruction.

4.4 Transfer functions and normalized coordinates

Let fx and fy be the frequency coordinates returned by `fftfreq` with one-pixel sampling. Both are in cycles/pixel, and D = sqrt(fx^2 + fy^2). For cutoff D0:

```
Ideal LPF:       H_L(D) = 1 if D <= D0, otherwise 0
Butterworth LPF: H_L(D) = 1 / (1 + (D/D0)^(2*n))
Gaussian LPF:    H_L(D) = exp(-D^2 / (2*D0^2))
Complement HPF:  H_H(D) = 1 - H_L(D)
```

The default is D0 = 0.10 cycles/pixel and n = 2. A horizontal frequency of 0.10 cycles/pixel has a period of 10 pixels. Each axis has Nyquist magnitude 0.5; diagonal radial frequency can approach sqrt(0.5^2 + 0.5^2), about 0.707. Therefore an ideal radial cutoff of 0.5 does not pass every diagonal frequency.

On a square unpadded image with N pixels per side, a radial bin distance r corresponds to D = r/N. After padding, recompute the frequency grid for the new shape. Keeping D0 in cycles/pixel preserves its physical interpretation. On a rectangular image, a circle in normalized frequency coordinates need not look circular if plotted using raw array indices with equal index spacing.

Equal D0 does not imply equal bandwidth. At D=D0, the ideal LPF still has gain 1 under the chosen inclusive boundary, Butterworth has gain 0.5, and Gaussian has gain exp(-1/2), approximately 0.6065. At D=0, every LPF has gain 1 and every HPF has gain 0. Larger Butterworth order sharpens its transition and can increase ringing. A Gaussian response changes smoothly and generally reduces ringing associated with abrupt spectral truncation.

4.5 Filtering and reconstruction

```
G_centered = H * F_centered
g_complex  = ifft2(ifftshift(G_centered))
g          = real(g_complex)
```

Real nonnegative masks attenuate components without changing the phase of nonzero retained coefficients. Taking only the displayed magnitude spectrum would discard phase and prevent correct reconstruction. The program checks the imaginary residual before accepting the real part.

A high-pass result contains positive and negative intensity deviations. Taking its absolute value changes the signal; clipping it immediately to unsigned 8-bit loses negative lobes. Raw arrays are therefore retained as floating-point values. PNG low-pass previews are clipped only for display; signed high-pass previews map zero to middle gray. Comparison plots use a diverging color map with an explicitly labeled scale.

For the same input, padding, and complementary masks:

```
g_low + g_high = original       (up to round-off)
g_sharp = original + alpha*g_high
        = (1+alpha)*original - alpha*g_low
```

The first identity is tested. The second produces high-boost sharpening with alpha >= 0, retaining the original background while emphasizing detail. It can also amplify noise and create overshoot. A high-pass residual alone is not a normal-looking sharpened photograph.

4.6 Boundary model

The DFT assumes a periodic extension. Multiplication of same-size spectra corresponds to circular convolution, so opposite borders can interact. The default pipeline reflects 32 pixels on every side before transforming and crops back after reconstruction. Reflection reduces sudden boundary jumps, but finite padding does not eliminate all long-range effects or ringing.

With a finite spatial convolution kernel of size P x Q, padding to at least (M+P-1) x (N+Q-1), positioning the kernel origin correctly, and cropping correctly can compute linear convolution with FFTs. This project's analytic frequency masks are defined on a finite DFT grid and do not all have a short spatial support. Its reflection strategy should not be described as an exact general conversion to linear convolution.

4.7 Worked numerical example

Consider an 8 x 8 image with every row equal to:

```
[120, 100, 80, 100, 120, 100, 80, 100]
f[x,y] = 100 + 20*cos(2*pi*2*y/8)
```

For this example only, use no padding and D0=0.25 cycles/pixel. The image contains a constant background of 100 and a horizontal sinusoid with amplitude 20 and period four pixels. Its only nonzero unshifted DFT coefficients are F[0,0]=6400 and the conjugate pair F[0,2]=F[0,6]=640, up to round-off. The DC value follows from 64 pixels times the mean of 100; each sinusoid coefficient is 64 times 20 divided by 2.

At the sinusoid frequency D=D0, each LPF keeps the mean but scales the sinusoid by its transfer gain. The HPF rejects DC and keeps the complementary amplitude:

Filter at D0	LPF gain	First four LPF pixels	First four HPF pixels
Ideal	1	120, 100, 80, 100	0, 0, 0, 0
Butterworth, n=2	0.5	110, 100, 90, 100	10, 0, -10, 0
Gaussian	0.606531	112.1306, 100, 87.8694, 100	7.8694, 0, -7.8694, 0

The remaining four columns repeat. Adding the low-pass and high-pass pixels recovers the original row. This example shows why a signed high-pass response is necessary and why equal numerical cutoffs do not produce identical filters. It also shows that filtering a noise-free image can remove useful information. The reusable tests check analytical sinusoidal gains separately from the noisy-image demonstration.

5. Algorithm and assignment coverage

Assignment step	Implemented operation	Evidence
1. Import libraries and load grayscale	NumPy/OpenCV/Matplotlib; grayscale PNG loading	Notebook step 1 (a-b)
2. Compute DFT	`fft2` on floating-point intensities	Notebook steps 2-4; core module
3. Center DC	`fftshift`; explicit frequency grid	Notebook steps 2-4
4. Analyze magnitude	`log1p(abs(F))` spectrum	Overview figure and observations
5. Apply LPF	Ideal, Butterworth, Gaussian masks	Core module and comparison figure
6. Apply HPF	Complement of each LPF	Signed outputs and DC tests
7. Apply IDFT	`ifftshift`, `ifft2`, residual check, crop	Core module and reconstruction tests
8. Compare images	Fixed-scale image panels and signed detail panels	Overview and comparison figures
9. Analyze effects	MSE, PSNR, cutoff study, row profiles	Results document and CSV
10. Document findings	Theory, observations, limitations, applications	Report and ten question answers

Pseudocode:

```
Load an 8-bit grayscale image; convert to float64.
For the controlled demo, retain an independent clean reference.
Reflect-pad the input by the chosen border width.
Compute its 2D FFT and center the spectrum.
Construct frequency coordinates in cycles/pixel.
For each filter family:
    Compute its low-pass mask and complementary high-pass mask.
    Multiply each mask by the complex centered spectrum.
    Undo the shift and apply the inverse FFT.
    Check imaginary round-off, keep the real part, and crop.
    Save signed float arrays and separately labeled display images.
Measure low-pass quality against the clean reference when available.
Compare families, inspect an edge profile, and vary the cutoff.
Verify reconstruction identities and document the observations.
```

6. Implementation walkthrough

`core.py` separates numerical operations from presentation. Validation rejects empty, non-finite, complex, or color arrays passed to the grayscale numerical API. It rejects invalid cutoffs, pass types, filter families, orders, and padding. Input arrays are copied so filtering does not modify caller data.

`frequency_mask` constructs a centered radial response from actual FFT sample frequencies. `filter_image` returns the cropped result, original centered spectrum, mask, filtered spectrum, padding, and maximum imaginary residual. `quality_metrics` evaluates MSE and PSNR directly on float arrays.

`opencv_round_trip` independently checks the transform convention. OpenCV's complex output uses two channels, so shifts specify `axes=(0,1)` and leave the channel axis intact. Its inverse uses `DFT_SCALE | DFT_REAL_OUTPUT` to include the required normalization. The OpenCV DFT API and tutorial provide the relevant transform operations.

`experiment.py` provides deterministic image generation, noise injection, comparison plots, cutoff measurements, PNG previews, compressed float arrays, and JSON provenance. `--input` loads an external image as supplied and omits full-reference metrics because no clean target is known. The notebook exposes the required operations step by step and verifies that they match the reusable implementation.

7. Evaluation protocol

All restoration metrics compare the noisy input or LPF reconstruction with the same clean reference, with intensity range L=255:

```
MSE  = mean((clean - estimate)^2)
PSNR = 10*log10(255^2 / MSE)
```

Lower MSE and higher PSNR indicate closer pixelwise reconstruction. If MSE is zero, PSNR is positive infinity; strict JSON records that value as null, with MSE=0 identifying the reason. The metric routine itself returns infinity.

Measurements use unclipped reconstructions, not PNG previews or independently normalized images. This includes overshoot in the error. PSNR is meaningful for same-scale reconstruction against a reference; it is not an appropriate quality score for a standalone signed high-pass residual.

Laplacian variance measures local second-derivative activity. Blur usually lowers it, but noise can raise it. Consequently a higher value is not evidence that useful edges have been preserved. Edge profiles and the visible thin line/texture provide complementary qualitative evidence. No SSIM, edge-detection accuracy, or runtime benchmark is claimed.

The cutoff sweep evaluates 0.03, 0.05, 0.08, 0.10, 0.15, 0.20, and 0.30 cycles/pixel for each family. Selecting a cutoff from this same reference is an illustrative, reference-assisted choice. A real evaluation should tune on separate validation data and report results on held-out images with multiple noise realizations.

8. Expected effects and interpretation

The centered spectrum usually has a bright central component because the image has nonzero mean and broad smooth regions. Paired peaks can reflect directional periodic texture. Gaussian noise spreads energy broadly; the spectrum is not a map that uniquely labels useful signal versus noise.

The LPF suppresses rapid fluctuations, making uniform and slowly varying areas smoother. It also attenuates true fine structure, rounds edges, and reduces texture contrast. A smaller cutoff applies stronger smoothing but can remove too much signal. A larger cutoff preserves more detail and more noise.

The HPF suppresses the mean and slowly varying background, leaving edges, texture, and high-frequency noise. Its zero-DC response gives zero mean over the full transformed padded domain; the cropped output need not have exactly zero mean. This distinction matters when checking reflected/cropped reconstructions.

An ideal cutoff abruptly truncates the spectrum and can create oscillatory ringing near transitions. Butterworth offers a tunable transition. Gaussian filtering trades a gradual response for smooth output. The best observed PSNR for one image should not be treated as a universal ranking of these families.

The actual measured results, figures, and experiment-specific conclusions are included in the following results section of the PDF and in `docs/RESULTS.md`.

9. Verification and limitations

Analytical sinusoidal tests check the gains at known frequencies, including the cutoff boundary. Constant images verify LPF DC preservation and HPF DC rejection. Complement tests cover even, odd, rectangular,

and single-row/single-pixel inputs with and without padding. A small direct circular-convolution calculation independently checks the convolution theorem implementation. Parseval's identity checks energy scaling. OpenCV and NumPy round trips check normalization.

The integration test runs the CLI on a decoded custom PNG, verifies saved artifacts and array shapes, and ensures no clean-reference scores are invented. Malformed data and invalid parameters produce actionable errors. The separate validation record states what was actually executed locally.

These tests reduce implementation risk; they do not certify universal correctness or guarantee a particular academic grade. The memory footprint is proportional to the padded pixel count with several complex arrays held at once. This is a clear teaching implementation, not a tiled or GPU-optimized large-image service. No claim is made that FFT filtering always runs faster than small spatial kernels.

10. Conclusion

The experiment realizes the full DFT-mask-IDFT workflow and demonstrates its central trade-off: removing high-frequency content can reduce noise while also weakening useful detail. Complementary HPFs isolate detail but retain noise; adding a scaled residual gives sharpening with its own noise and overshoot cost. Filter shape, cutoff, boundary handling, and evaluation method jointly determine whether a result is useful. The generated measurements support conclusions for the controlled demo, while the reusable implementation permits further testing on other images.

Measured results and observations

Generated from `results/demo/metrics.json` and `cutoff_sweep.csv`; the figures show the same run.

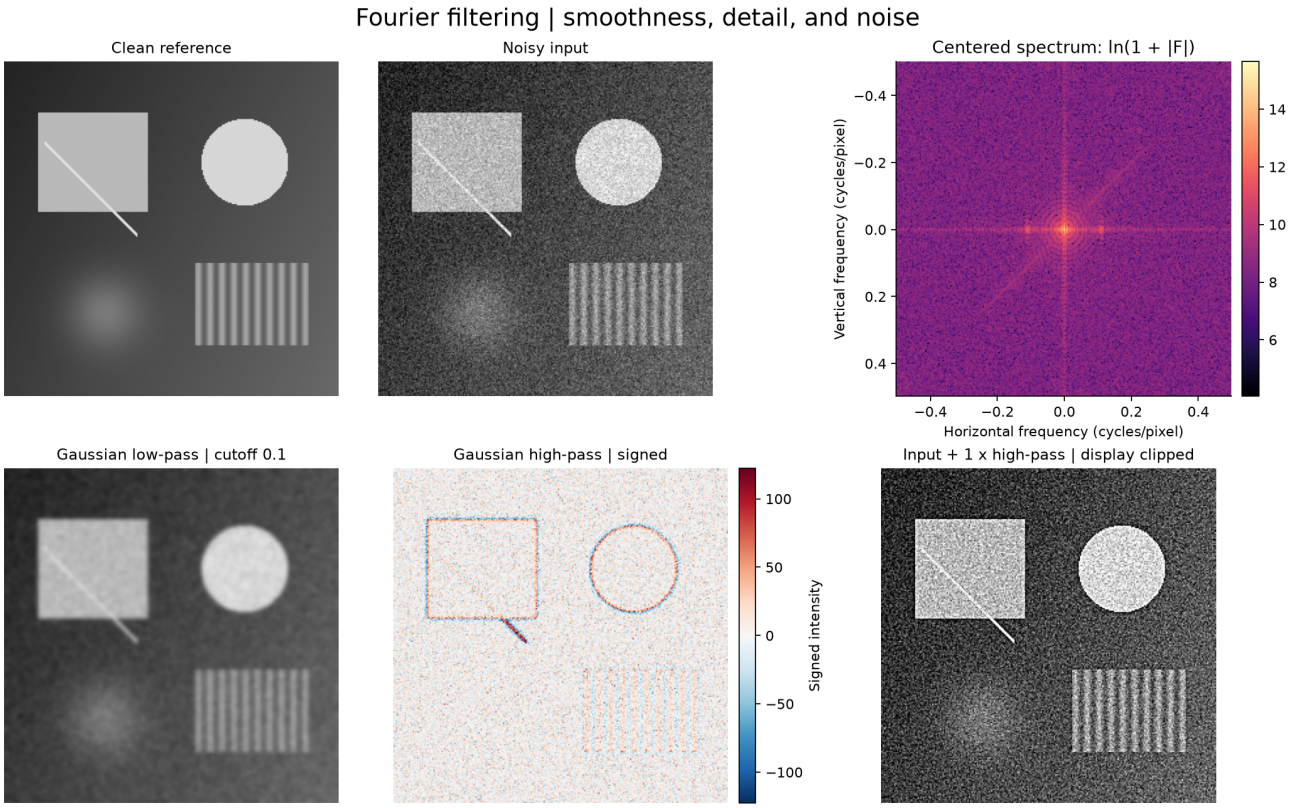
Experiment configuration

Image: 256 x 256 grayscale; seed: 42; noise sigma before clipping: 20 intensity units. Radial cutoff: 0.1 cycles/pixel; Butterworth order: 2; reflect padding: 32 pixels per side.

Reconstruction quality

Image	MSE	PSNR (dB)	Laplacian variance
Original	400.1924	22.1081	8572.9356
Ideal Low	158.2923	26.1362	4.6949
Butterworth Low	105.4073	27.9021	6.5745
Gaussian Low	90.1942	28.5790	12.4723

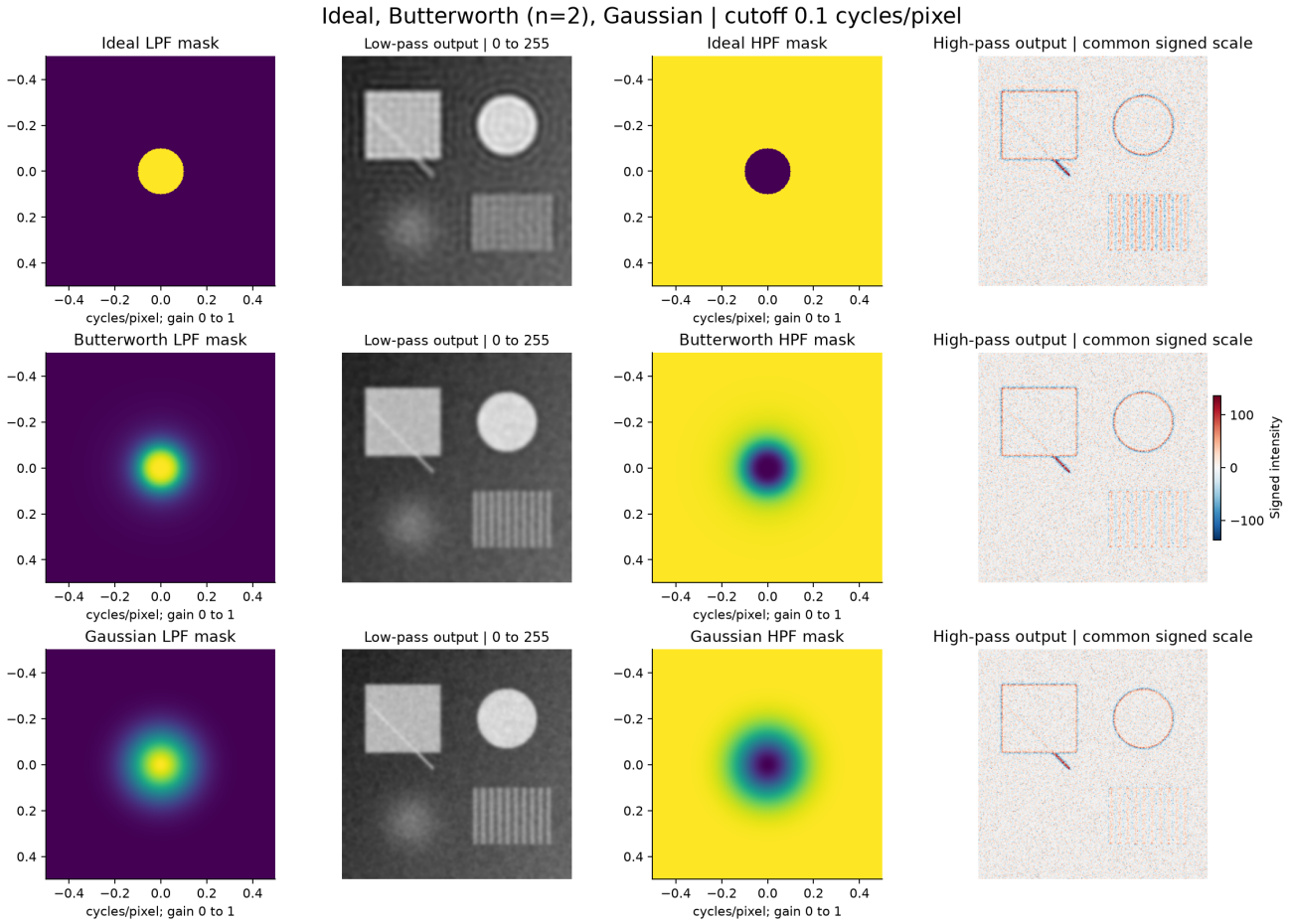
At this cutoff, Gaussian LPF changes PSNR by +6.4709 dB and reduces MSE by 77.46% relative to the noisy input. These measurements use raw float reconstructions before clipping. Laplacian variance is descriptive: its decrease reflects less rapid variation, including both noise suppression and loss of true detail.



Overview of Fourier filtering

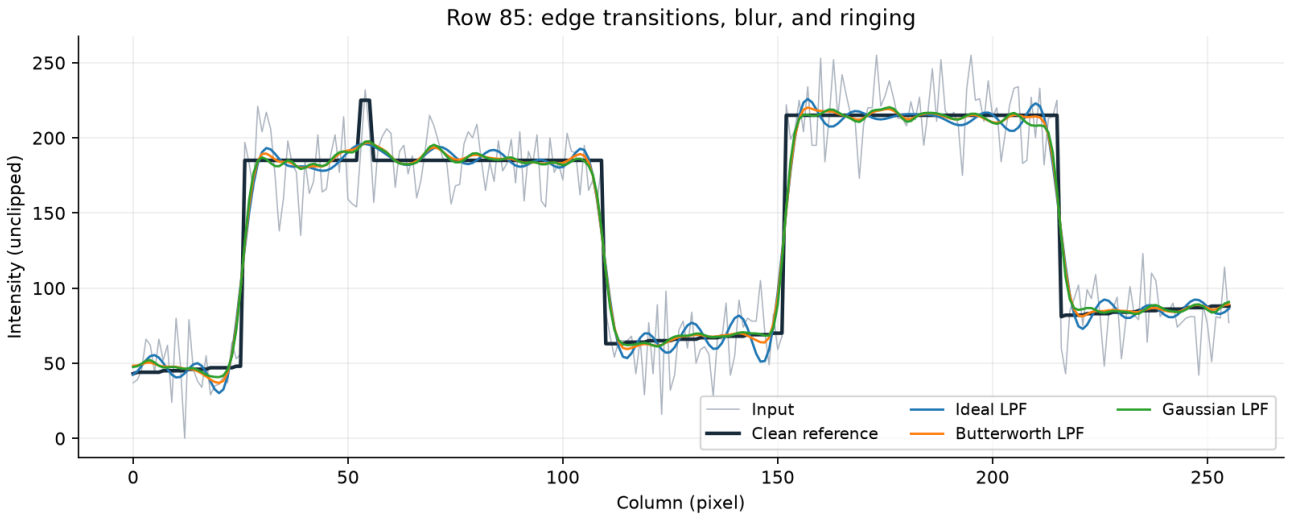
The low-pass panel smooths the noisy background, but fine structures and stripe contrast are weakened. The signed high-pass panel contains object boundaries, texture, and noise. The high-boost panel adds that residual back to the input; its apparent sharpness should not be interpreted as evidence of noise removal.

Filter shape and edge profiles



Comparison of all filters

LPF panels share a 0..255 display range. HPF comparison panels share a symmetric signed scale. The masks have equal numerical cutoffs but different responses at that cutoff, so their effective bandwidths differ.

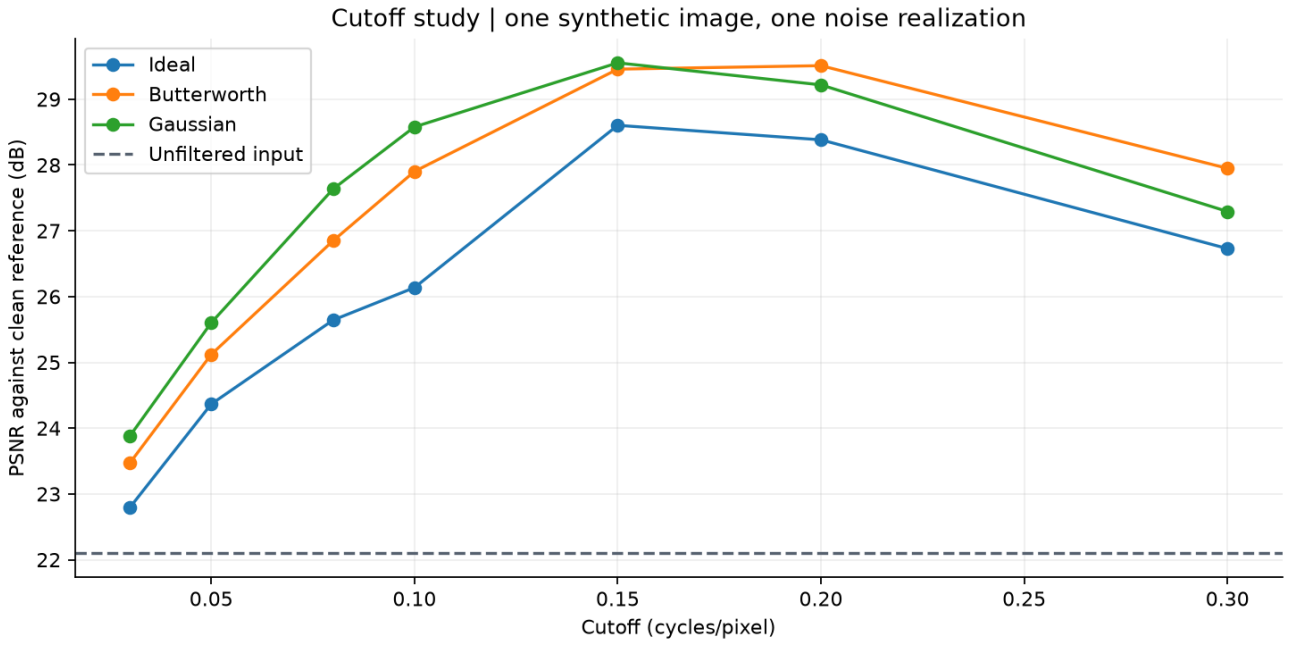


Unclipped row profile

The profile exposes the trade-off hidden by a simple image preview: noise fluctuations decrease while transitions spread. The ideal filter's abrupt cutoff can produce oscillation near boundaries; increasing Butterworth order makes its response more abrupt. The Gaussian filter has a smoother transition. Profiles retain overshoot that PNG clipping could hide.

Cutoff sensitivity

Family	Best tested cutoff (cycles/pixel)	Best tested PSNR (dB)
Ideal	0.15	28.6031
Butterworth	0.20	29.5099
Gaussian	0.15	29.5528



Cutoff study

These are maxima over seven sampled cutoffs, not continuous optima. The clean reference is used to inspect the trade-off, so the selection is reference-assisted. It is not a held-out evaluation or a universal recommended cutoff. Stronger filtering can erase signal; weaker filtering retains more noise. Both can increase reconstruction error.

Numerical diagnostics

Diagnostic	Maximum absolute value
numpy round trip max abs error	8.527e-14
opencv round trip max abs error	1.137e-13
opencv numpy dft max abs difference	2.346e-10
max imaginary residual	5.409e-14
Ideal LPF + HPF - input	1.421e-13
Butterworth LPF + HPF - input	1.421e-13
Gaussian LPF + HPF - input	1.705e-13

The small residuals support the implemented transform, normalization, and complement identities. The unnormalized forward DFT has larger coefficient scales than pixel intensities, so its absolute cross-library difference should not be compared directly to a pixel reconstruction error.

Reproducibility

Recorded software: python 3.11.0, numpy 2.4.6, opencv 4.14.0, matplotlib 3.11.1. The input hash is stored in metrics.json. Tiny round-off differences can occur across platforms or library versions; test tolerances account for this. No performance benchmark or downstream feature accuracy was measured.

Assignment questions: complete answers

Q1. What is the Fourier Transform? Why is it important in digital image processing?

The Fourier Transform expresses a signal as a combination of sinusoidal components with different frequencies, amplitudes, and phases. In a digital image, intensity is sampled along two spatial axes, so the two-dimensional DFT produces a finite array of complex frequency coefficients. The FFT efficiently computes those coefficients.

It is important because structures that are mixed together in pixel space may have distinguishable frequency characteristics. Broad shading is often concentrated at low frequencies, fine texture occupies higher or directional bands, and periodic interference can produce isolated conjugate peaks. A filter can attenuate selected components before reconstructing the image.

The transform also gives a mathematical link between convolution and spectral multiplication. This supports efficient large-kernel filtering and provides a framework for blur modeling, restoration, spectral texture analysis, and image registration. Magnitude alone is insufficient: phase contains spatial alignment information and must be preserved for faithful reconstruction. Fourier filtering is most useful when the desired operation or corruption has a meaningful frequency-domain description.

Q2. Differentiate between the spatial domain and the frequency domain.

Aspect	Spatial domain	Frequency domain
Representation	Pixel intensities f[x,y]	Complex coefficients F[u,v]
Coordinates	Row and column positions	Horizontal and vertical spatial frequencies
Typical operation	Point mapping or neighborhood operation	Modification of spectral amplitude and phase
Examples	Thresholding, median filtering, small Sobel kernels	Low/high-pass masks, notch rejection, Wiener restoration
Interpretation	Where a feature occurs	Which scales and directions contribute
Locality	Neighborhood operations can be explicitly local	A DFT coefficient describes a basis pattern across the whole image
Main trade-off	Simple local processing; large kernels may cost more	Transform overhead, global boundary model, and complex arithmetic

The domains are alternative representations of the same sampled image when the complete DFT is retained. A linear shift-invariant spatial convolution can be represented as multiplication by the kernel's transfer function, provided boundary conventions and kernel alignment agree. Nonlinear operations such as median filtering are not generally equivalent to a single fixed frequency mask.

For example, a small spatial Gaussian blur is convenient for light smoothing. A pair of frequency notches can specifically remove visible periodic striping when its frequencies are known. The choice depends on the operation rather than on one domain being universally superior.

Q3. What is the significance of the Discrete Fourier Transform in image processing?

The DFT makes Fourier analysis directly applicable to finite sampled images. It maps M x N pixel values to M x N complex coefficients and is invertible when all coefficients are retained with a consistent normalization convention.

Its main contributions are frequency analysis, filtering, system modeling, and efficient computation. The magnitude spectrum reveals concentration of energy, dominant scales, and periodic patterns. Multiplying coefficients by a transfer function implements a linear filtering operation. A blur can be represented by its optical or computational transfer function, enabling restoration methods that account for attenuation of different frequencies.

The DFT also exposes useful numerical properties. The DC coefficient gives total intensity, conjugate symmetry follows from real input, and Parseval's identity relates spatial energy to frequency energy. These properties provide strong implementation checks. FFT algorithms reduce the computational burden without changing the mathematical DFT being evaluated.

In this solution, a complete unfiltered DFT/IDFT pair reconstructs the original within floating-point precision. Applying a mask deliberately changes some coefficients, so the filtered reconstruction differs from the original by design. Attenuated or removed information cannot be assumed recoverable merely because an inverse transform exists.

Q4. Explain the purpose of shifting the zero-frequency component to the center of the frequency spectrum.

Standard DFT storage places the zero-frequency or DC coefficient at the first array element. Positive and negative frequencies then occupy different parts of each axis. `fftshift` rearranges the array so negative frequencies, DC, and positive frequencies are easier to interpret visually, with DC near the center.

This layout makes it convenient to inspect low-frequency concentration and construct centered radial masks. A central low-pass region preserves broad variations, while its complement removes them. The shift changes only the order of coefficients; it does not change their values, compute a new transform, enhance the image, or perform filtering.

Before `ifft2`, the rearrangement must be undone using `ifftshift`. Applying `fftshift` a second time is not a generally correct inverse for odd-length axes. In OpenCV, complex arrays have a real/imaginary channel dimension, so shifting should specify only the spatial axes. The notebook and tests explicitly verify these conventions on both odd and even dimensions. The NumPy inverse-shift API defines the inverse operation.

Q5. Compare Low-Pass Frequency Filters and High-Pass Frequency Filters with suitable applications.

Property	Low-pass filter	High-pass filter
Passed components	Frequencies near DC	Frequencies away from DC
Attenuated components	Rapid intensity variation	Constant and slowly varying background
Typical visible effect	Smoothing and blur	Signed edge and texture residual
DC response in this project	1: preserves the full-domain mean	0: rejects the full-domain mean
Benefits	Suppresses some high-frequency noise	Emphasizes transitions and fine structure
Main costs	Loss of detail and edge sharpness	Retention of noise; possible halos
Suitable uses	Pre-smoothing, anti-alias filtering before downsampling	Detail extraction, background suppression, high-boost sharpening

An LPF can improve an image corrupted by broadband noise when much of the useful signal lies in lower frequencies. Anti-alias filtering must be designed for the new sampling rate; an arbitrary blur or arbitrary D0 is not automatically adequate. The sampling/downsampling operation itself is separate from filtering.

An HPF can reveal local defects against smooth illumination or supply a detail residual for sharpening. For a complementary pair, H_H=1-H_L. Sharpening is `original + alpha*high_pass`, while the high-pass residual by itself lacks the original background and average brightness.

The masks implemented here have gains between zero and one, so they do not amplify the absolute amplitude of any Fourier component by themselves. Noise can become more visible relative to the removed background. Adding the residual for high-boost sharpening can amplify both fine detail and noise.

Ideal, Butterworth, and Gaussian describe response shapes, not separate goals. Ideal masks give sharp spectral selection with ringing risk; Butterworth offers a controllable roll-off; Gaussian gives a smooth roll-off. The appropriate type depends on the noise spectrum, useful feature scale, and acceptable artifacts.

Q6. What is the role of the Inverse Fourier Transform in image reconstruction?

The IDFT combines complex frequency coefficients back into spatial pixel values. If the full unmodified spectrum is retained, it inverts the DFT and recovers the input up to numerical precision. If coefficients have been filtered, it instead reconstructs the spatial image represented by that modified spectrum.

The correct sequence is to multiply the centered complex spectrum by the mask, undo the shift, and apply the inverse transform with the matching normalization. NumPy's `ifft2` includes the factor 1/(M*N). OpenCV's inverse needs the scaling flag when using its otherwise unscaled convention. Reflect padding is removed after inversion to recover the original output dimensions.

For the real input and symmetric real masks used here, the imaginary part should be negligible. A significant imaginary residual suggests a symmetry, channel, or transform-handling error and should be investigated. The signed real part is the reconstructed signal. Taking its magnitude or independently normalizing its range changes that signal and should be reserved for explicitly labeled views.

The IDFT does not undo information loss caused by a zero-valued filter response. It faithfully synthesizes the remaining frequencies. Thus reconstruction after low-pass filtering is a smoother image, not recovery of the discarded edges.

Q7. Why is frequency domain filtering preferred for certain image enhancement tasks?

It is useful when the task can be expressed naturally as selective modification of frequencies or when large linear filters make transform-based computation attractive. For example, periodic interference may appear as narrow spectral peaks; notch filters can target those peaks more directly than a generic blur. A known blur transfer function also gives a natural basis for restoration.

The frequency response exposes exactly which scales are passed, attenuated, or amplified. Smooth transfer functions permit controlled transitions, and several linear transfer functions can be multiplied before one inverse transform when their assumptions and boundary models agree. Large-kernel operations may gain from FFT-based convolution because the transform cost grows roughly as pixel count times its logarithm.

These advantages are conditional. Small spatial kernels often cost less and are easier to deploy. Sharp spectral masks can ring, FFTs impose a global periodic model unless boundaries are handled, and simple global masks cannot distinguish signal and noise occupying the same frequencies. Nonlinear edge-preserving or impulse-noise methods may be more appropriate for their respective tasks.

Accordingly, this experiment demonstrates frequency selectivity and its trade-offs. It does not assume that the FFT is always faster or that frequency filtering always improves every image.

Q8. Mention four real-world applications where Fourier Transform is used in computer vision and image analysis.

1. **Periodic artifact removal in scanned or remotely sensed images.** Regular striping or repeated interference can produce conjugate spectral peaks. A carefully designed notch-reject mask attenuates these frequencies while preserving more of the surrounding spectrum. The mask must preserve the symmetry needed for real reconstruction and may still remove overlapping legitimate texture.

2. **Blur restoration in microscopy and imaging systems.** A point-spread function can be transformed into a transfer function. Regularized inversion or Wiener filtering then balances reversal of blur with noise sensitivity. Frequencies strongly suppressed by the imaging system cannot be recovered reliably by unrestricted division.

3. **Translation estimation and image registration.** A spatial translation creates a phase change in the Fourier domain. Phase correlation uses a normalized cross-power spectrum and an inverse transform to locate a shift peak, supporting frame alignment and motion estimation. Windowing, overlap, texture, and subpixel estimation affect real performance. OpenCV documents phase correlation.

4. **Texture and orientation analysis.** Spectral peaks and band energies can characterize repeated fabric patterns, manufactured surface textures, or ridge-like structures. Deviations may support inspection or feature construction. Local or windowed analysis is often needed because a global Fourier spectrum does not directly identify the location of a defect.

These are applications of the Fourier framework. The current code implements radial low/high-pass filtering and high-boost illustration; it does not claim to implement notch filtering, deconvolution, registration, or a complete texture classifier.

Q9. Compare frequency domain filtering with spatial domain filtering based on computational efficiency and practical applications.

Let P=M*N be the image pixel count and K the number of coefficients in a dense spatial kernel. Direct convolution costs approximately O(P*K). For a k x k kernel this is O(P*k^2); if the kernel is separable, two one-dimensional passes reduce it to approximately O(P*k). Some special filters have additional optimizations, such as running sums or integral images for box filtering.

FFT filtering on a padded grid containing P' pixels costs approximately O(P'*log(P')) for forward and inverse transforms plus O(P') for spectral multiplication. Computing or transforming a new kernel adds work; a

reused kernel spectrum can be cached. Memory use increases because padded real and complex arrays must be stored. Transform constants, array dimensions, hardware, threading, and implementation determine the practical crossover.

Situation	Often suitable approach	Reason
Small 3 x 3 or 5 x 5 neighborhood	Spatial filtering	Low overhead and good locality
Separable Gaussian smoothing	Two spatial 1D passes	Exploits separability
Very large dense linear kernel	FFT convolution	Favorable asymptotic cost
Narrow periodic interference	Frequency notch filter	Direct spectral targeting
Salt-and-pepper corruption	Spatial median filter	Nonlinear rejection of outliers
Local edge-preserving smoothing	Suitable nonlinear/local method	Content-dependent behavior

An equivalent comparison must use the same operation, kernel, image size, precision, boundary model, and output crop. Otherwise different images or different runtimes may reflect different operations. Notably, OpenCV's `filter2D` is a correlation API and may itself choose a DFT implementation for larger kernels; calling it does not prove that a purely direct algorithm was used. See the OpenCV filtering reference.

No runtime benchmark is supplied here, so no measured speedup is asserted. The defensible conclusion is that FFT methods are attractive for large or spectrally specific linear operations, whereas small or nonlinear neighborhood operations commonly favor spatial methods.

Q10. How does frequency domain filtering improve the performance of image restoration and feature extraction techniques?

For restoration, an observed image can be modeled as a clean image blurred by a point-spread function and corrupted by additive noise. Under a consistent convolution model, the frequency-domain expression is:

```
Y[u,v] = B[u,v]*X[u,v] + N[u,v]
```

Here B is the blur transfer function, X the clean spectrum, N noise, and Y the observation. Where signal and interference occupy sufficiently distinct bands, selective attenuation can improve signal-to-noise ratio. Periodic noise is a good candidate for narrow rejection. A simple LPF is useful only when the detail loss it introduces is outweighed by reduced noise.

Naive inverse restoration uses Y/B, but small or zero B values cause unstable noise amplification or undefined division. A common Wiener-form estimate uses:

```
X_hat = [conjugate(B) / (abs(B)^2 + S_n/S_x)] * Y
```

S_n and S_x are noise and signal power spectral densities. This expresses the balance between inversion and noise suppression, under the model assumptions and available spectral estimates. It is background theory, not a claim that this repository implements Wiener filtering.

For feature extraction, suppressing slow illumination variation or emphasizing selected frequency bands can make relevant texture or transitions easier for later analysis. However, HPF output is not automatically an edge map: a detector may still need gradient direction, thresholding, nonmaximum suppression, and validation. Strong noise or ringing can create false features.

Improvement should be evaluated for the actual objective. Restoration can use reference-based MSE/PSNR and visual checks; downstream extraction should use task metrics such as boundary accuracy

or classification performance on held-out data. The current experiment measures reconstruction error for denoising and qualitatively examines detail. It does not claim improved detector accuracy.

Appendix A - Runnable reference program

Complete listing of examples/basic_solution.py. Generate the demo first, then run this file from the repository. It directly implements the required Gaussian LPF/HPF workflow using the three assigned libraries. The reusable package adds filter families, evaluation and validation.

```
"""Minimal standalone assignment workflow. Run after generating results/demo.
Usage: python examples/basic_solution.py
This listing uses NumPy/OpenCV/Matplotlib directly, without importing the package.
"""
from pathlib import Path
import cv2
import matplotlib
import numpy as np
matplotlib.use("Agg")
import matplotlib.pyplot as plt
def main() -> None:
    root = Path(__file__).resolve().parents[1]
    source = root / "results/demo/original.png"
    if not source.is_file():
        raise FileNotFoundError("Run python -m frequency_filtering first")
    encoded = np.fromfile(source, dtype=np.uint8)
    decoded = cv2.imdecode(encoded, cv2.IMREAD_GRAYSCALE)
    if decoded is None:
        raise ValueError("The input could not be decoded as grayscale")
    image = decoded.astype(np.float64)
    cutoff, border = 0.10, 32
    padded = np.pad(image, border, mode="reflect")
    spectrum = np.fft.fftshift(np.fft.fft2(padded))
    fy = np.fft.fftshift(np.fft.fftfreq(padded.shape[0]))[:, None]
    fx = np.fft.fftshift(np.fft.fftfreq(padded.shape[1]))[None, :]
    distance = np.hypot(fy, fx)
    low_mask = np.exp(-(distance**2) / (2 * cutoff**2))
    high_mask = 1.0 - low_mask

    crop = (
        slice(border, border + image.shape[0]),
        slice(border, border + image.shape[1]),
    )
    outputs = []
    for mask in (low_mask, high_mask):
        inverse = np.fft.ifft2(np.fft.ifftshift(spectrum * mask))
        if np.max(np.abs(inverse.imag)) > 1e-10:
            raise ArithmeticError("Unexpected imaginary residual")
        outputs.append(inverse.real[crop])
    low, high = outputs
    np.testing.assert_allclose(low + high, image, rtol=0, atol=1e-10)
    fig, axes = plt.subplots(2, 2, figsize=(10, 8), layout="constrained")
    axes[0, 0].imshow(image, cmap="gray", vmin=0, vmax=255)
    axes[0, 0].set_title("Original grayscale input")
    axes[0, 1].imshow(np.log1p(np.abs(spectrum)), cmap="magma")
    axes[0, 1].set_title("Padded centered spectrum: ln(1 + |F|)")
    axes[1, 0].imshow(low, cmap="gray", vmin=0, vmax=255)
    axes[1, 0].set_title("Gaussian low-pass reconstruction")
    bound = max(float(np.max(np.abs(high))), 1e-12)
    im = axes[1, 1].imshow(high, cmap="RdBu_r", vmin=-bound, vmax=bound)
    axes[1, 1].set_title("Gaussian high-pass: signed detail")
    fig.colorbar(im, ax=axes[1, 1], label="Signed intensity")
    for ax in axes.flat:
        ax.axis("off")
    destination = root / "results/basic"
    destination.mkdir(parents=True, exist_ok=True)
    fig.savefig(destination / "comparison.png", dpi=150)
```

```
    plt.close(fig)
    np.savez_compressed(destination / "arrays.npz", original=image, low=low, high=high)
    print(f"Saved comparison and signed arrays to {destination}")
if __name__ == "__main__":
    main()
```

Appendix B - Verification and submission

Local verification record

Verified on 2026-09-09 on Windows with Python 3.11.0. Exact direct dependency versions are recorded in `requirements-tested.txt`; numerical-run versions are also embedded in `results/demo/metrics.json`.

Check	Observed result
`python -m pytest`	86 tests passed
`python -m ruff check .`	Passed
`python -m ruff format --check .`	Passed
`python -m pip check`	No broken requirements
Default command-line experiment	Completed; figures, JSON, CSV and NPZ generated
Custom-image command-line integration	Passed, including omission of unavailable reference metrics
Executed notebook	All 7 code cells completed; 6 embedded PNG outputs; no error outputs
Compact reference example	Completed; Gaussian LPF/HPF comparison and raw arrays generated
`python -m build --no-isolation`	Source distribution and wheel built successfully

The test suite includes analytical frequency gains, DC behavior, complement reconstruction, circular convolution, Parseval energy, OpenCV/NumPy agreement, odd/even dimensions, malformed images and parameters, reproducibility, and CLI artifacts. The notebook independently applies the Gaussian mask using both NumPy and OpenCV and checks that their reconstructed images agree.

The review also checks the smallest positive cutoff, diagonal Nyquist response, frequency-bin alignment in plots, a single-pixel input, zero-noise infinite PSNR serialization, and invalid saved metadata. Reconstruction comparisons use zero relative tolerance and explicit absolute tolerances, so the tolerance does not grow with image intensity. The worked 8 x 8 example was independently evaluated: its nonzero DFT coefficients are 6400, 640, and 640; its reported filter outputs agree with direct execution.

The curated overview, filter comparison, line profile, and cutoff plots were visually inspected. The PDF is generated from the same written solution, measured tables, figures, and compact reference source listing.

The original two-page assignment was inspected directly. All ten required steps are mapped in the laboratory report, and Questions 1-10 are answered in the written solution and executed notebook. No grading rubric beyond that supplied assignment was assumed.

The CI workflow is configured for Windows and Ubuntu with Python 3.11 and 3.12. Those hosted runs have not occurred locally; their status will be available in GitHub Actions after the repository is pushed. No cross-platform performance benchmark, clinical evaluation, or downstream feature-extraction accuracy is claimed.

GitHub submission guide

The solution is ready to review and publish as a repository. No GitHub account, remote URL, identity, or student details were supplied, so publication is left to you. Local files contain no invented name, college, or roll number.

For a single-file submission, use `output/BTCS301P-AS04.zip`. Its contents are the complete project and a SHA-256 inventory. Regenerate it after your edits with `python scripts/package_submission.py`. For GitHub, upload or push the extracted project files so that the README, notebook, and tests are usable.

Review before publishing

1. Read `README.md`, the notebook, and the full report. Be prepared to explain the code, cutoff choice, signed HPF output, and measured observations.

2. Add any required student details to `docs/LAB_REPORT.md` and rebuild the PDF.

3. Run `python -m pytest` and `python -m ruff check .`.

4. Keep the notebook outputs, `results/demo`, and final PDF in the repository. The `.venv`, caches, temporary files, custom runs, and submission ZIP are ignored.

5. Review `git status --short` and `git diff --cached` before committing. The original assignment PDF is outside the repository and is not staged.

Create and push the repository

Create an empty repository on GitHub named `BTCS301P-AS04`. Leave GitHub's automatic README, license, and gitignore options unchecked because project files already exist. Choose public or private according to your submission requirements. No license grant has been assumed; add one if you want to grant reuse rights under a specific license.

Run these commands from this project folder. If `git status` already works, skip `git init`. Replace `YOUR_USERNAME` with your actual GitHub username:

```
git init
git add .
git diff --cached --stat
git commit -m "Complete Fourier image filtering experiment with report and tests"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/BTCS301P-AS04.git
git push -u origin main
```

Git may prompt for an author name/email if these are not configured. Set your own values; do not copy someone else's identity. Authenticate using your normal GitHub sign-in or credential manager. Do not place tokens in source files or notebook cells. If `origin` already exists, inspect `git remote -v` before making changes instead of adding it again.

After pushing, open the repository on GitHub and verify that the overview image, notebook outputs, report link, and file paths render. Review the Actions tab for the hosted test results. The workflow's first run occurs only after publication.